

MÓDULO 7 — Dashboard Visual y Gestión de Usuarios

Objetivo: Implementar indicadores clave de rendimiento (*Key Performance Indicator, KPI*), y una interfaz profesional para administrar los permisos de los usuarios registrados.

En este **MÓDULO 7** se transforma la aplicación para crear un centro de control del sistema de tickets. El Administrador obtiene una visión global del sistema y toma decisiones sobre el equipo técnico.

1. Preparación de la Lógica del Servicio (*TicketService.cs*)

Para que el Dashboard sea rápido, se van a realizar consultas agregadas de LINQ que se ejecuten directamente en SQL Server.

Editar el Archivo: */Services/TicketService.cs* (Actualización del código)

```
1  using Microsoft.EntityFrameworkCore;
2  using SupportTicketsSystem2026.Data;
3  using SupportTicketsSystem2026.Enums;
4  using SupportTicketsSystem2026.Models;
5  using System.Security.Claims;
6
7  namespace SupportTicketsSystem2026.Services; // Recuerde que debe usar su propio namespace
8
9  // Servicio para manejar la lógica del servicio relacionada con los tickets de soporte
10 // Usando Primary Constructor de C# 14
11 public class TicketService(IDbContextFactory<ApplicationDbContext> dbFactory)
12 {
13     // --- MÓDULO 4: MÉTODOS DE CREACIÓN ---
14
15     1 reference
16     public async Task<List<Category>> GetCategoriesAsync()
17     {
18         using var context = dbFactory.CreateDbContext();
19         return await context.Categories.ToListAsync();
20     }
21
22     1 reference
23     public async Task CreateTicketAsync(Ticket ticket)
24     {
25         using var context = dbFactory.CreateDbContext();
26         context.Tickets.Add(ticket);
27         await context.SaveChangesAsync();
28     }
29
30     // --- MÓDULO 5: SEGURIDAD Y FILTRADO ---
31     1 reference
32     public async Task<List<Ticket>> GetTicketsForUserAsync(ClaimsPrincipal user)
33     {
34         using var context = dbFactory.CreateDbContext();
35         var userId = user.FindFirst(ClaimTypes.NameIdentifier)?.Value;
36
37         var query = context.Tickets
38             .Include(t => t.Category)
39             .Include(t => t.Author)
40             .Include(t => t.Technician)
41             .AsQueryable();
42
43         if (user.IsInRole(Roles.Admin))
44         {
45             // El admin ve todo
46             return await query.ToListAsync();
47         }
48         else if (user.IsInRole(Roles.Technician))
49         {
50             return await query.Where(t => t.TechnicianId == userId ||
51                                     t.Status == Enums.TicketStatus.Open).ToListAsync();
52         }
53         // El empleado solo ve los tickets que él mismo creó
54         return await query.Where(t => t.AuthorId == userId).ToListAsync();
55     }
56
57     // --- MÓDULO 6: GESTIÓN ADMINISTRATIVA ---
58     // Método para que el admin pueda ver todos los tickets sin restricciones
59     1 reference
60     public async Task<List<Ticket>> GetAllTicketsAsync()
61     {
62         using var context = dbFactory.CreateDbContext();
63         return await context.Tickets
64             .Include(t => t.Category)
65             .Include(t => t.Author)
66             .Include(t => t.Technician)
67             .OrderByDescending(t => t.CreatedAt)
68             .ToListAsync();
69     }
```

```

66 // Método para actualizar un ticket (usado por admin o técnico)
67 public async Task UpdateTicketAsync(Ticket ticket)
68 {
69     using var context = dbFactory.CreateDbContext();
70     ticket.UpdatedAt = DateTime.UtcNow;
71     context.Entry(ticket).State = EntityState.Modified;
72     await context.SaveChangesAsync();
73 }
74 // Método para eliminar un ticket (usado por admin)
75 public async Task<List<ApplicationUser>> GetUsersByRoleAsync(string roleName)
76 {
77     using var context = dbFactory.CreateDbContext();
78
79     // Buscamos el ID del rol primero
80     var role = await context.Roles.FirstOrDefaultAsync(r => r.Name == roleName);
81     if (role == null) return new List<ApplicationUser>();
82
83     // Buscamos los IDs de usuarios asociados a ese rol
84     var userIds = await context.UserRoles
85         .Where(ur => ur.RoleId == role.Id)
86         .Select(ur => ur.UserId)
87         .ToListAsync();
88
89     // Retornamos la lista de objetos de usuario
90     return await context.Users
91         .Where(u => userIds.Contains(u.Id))
92         .ToListAsync();
93 }
94
95 // Obtener un ticket específico con todos sus detalles y relaciones
96 public async Task<Ticket?> GetTicketByIdAsync(int id)
97 {
98     using var context = dbFactory.CreateDbContext();
99     return await context.Tickets
100         .Include(t => t.Category)
101         .Include(t => t.Author)
102         .Include(t => t.Technician)
103         .FirstOrDefaultAsync(t => t.Id == id);
104 }
105
106 // Listar todos los técnicos disponibles para asignar
107 public async Task<List<ApplicationUser>> GetTechniciansAsync()
108 {
109     using var context = dbFactory.CreateDbContext();
110     var techRole = await context.Roles.FirstOrDefaultAsync(r => r.Name == Roles.Technician);
111     if (techRole == null) return [];
112
113     var techIds = await context.UserRoles
114         .Where(ur => ur.RoleId == techRole.Id)
115         .Select(ur => ur.UserId).ToListAsync();
116
117     return await context.Users.Where(u => techIds.Contains(u.Id)).ToListAsync();
118 }
119
120 // Actualizar el ticket (Asignación o Cambio de Estado)
121 public async Task UpdateTicketStatusAsync(int ticketId, TicketStatus newStatus, string? technicianId = null)
122 {
123     using var context = dbFactory.CreateDbContext();
124     var ticket = await context.Tickets.FindAsync(ticketId);
125     if (ticket != null)
126     {
127         ticket.Status = newStatus;
128         if (!string.IsNullOrEmpty(technicianId)) ticket.TechnicianId = technicianId;
129         ticket.UpdatedAt = DateTime.UtcNow;
130
131         await context.SaveChangesAsync();
132     }
133 }
134
135 // --- MÓDULO 7: MÉTODOS DE REPORTE Y ESTADÍSTICAS ---
136 public async Task<Dictionary<string, int>> GetTicketStatsAsync()
137 {
138     using var context = dbFactory.CreateDbContext();
139     return new Dictionary<string, int>
140     {
141         ["Total"] = await context.Tickets.CountAsync(),
142         ["Open"] = await context.Tickets.CountAsync(t => t.Status == TicketStatus.Open),
143         ["InProgress"] = await context.Tickets.CountAsync(t => t.Status == TicketStatus.InProgress),
144         ["Resolved"] = await context.Tickets.CountAsync(t => t.Status == TicketStatus.Resolved),
145         ["Urgent"] = await context.Tickets.CountAsync(t => t.Priority == TicketPriority.Urgent)
146     };
147 }
148
149 public async Task<List<ApplicationUser>> GetAllUsersAsync()
150 {
151     using var context = dbFactory.CreateDbContext();
152     return await context.Users.OrderBy(u => u.Email).ToListAsync();
153 }
154 }
155

```



2. Dashboard con Estadísticas Visuales

Actualizamos la página de inicio del administrador para que sea informativa. Utilizaremos las "Cards" de Bootstrap para resaltar los números críticos.

Editar el Archivo: /Components/Pages/Admin/Dashboard.razor

```
1  @page "/admin/dashboard"
2  @using Microsoft.AspNetCore.Authorization
3  @using SupportTicketsSystem2026.Data
4  @using SupportTicketsSystem2026.Models
5  @using SupportTicketsSystem2026.Services
6  @using SupportTicketsSystem2026.Enums
7  @inject TicketService TicketService
8  @inject NavigationManager Navigation
9  @attribute [Authorize(Roles = Roles.Admin)]
10 @rendermode InteractiveServer
11
12 <PageTitle>Admin Dashboard - System Overview</PageTitle>
13
14 <div class="container-fluid mt-4">
15     <div class="d-flex justify-content-between align-items-center mb-4">
16         <h3><i class="bi bi-speedometer2"></i> Administrative Dashboard</h3>
17         <span class="text-muted">Real-time Support Metrics</span>
18     </div>
19
20     @* --- SECCIÓN DE ESTADÍSTICAS (MÓDULO 7) --- *@
21     <div class="row g-3 mb-5">
22         <div class="col-md-3">
23             <div class="card bg-primary text-white border-0 shadow-sm">
24                 <div class="card-body">
25                     <h6 class="card-title text-uppercase opacity-75 small">Total Tickets</h6>
26                     <h2 class="display-6 fw-bold">@stats["Total"]</h2>
27                 </div>
28             </div>
29
30             <div class="col-md-3">
31                 <div class="card bg-warning text-dark border-0 shadow-sm">
32                     <div class="card-body">
33                         <h6 class="card-title text-uppercase opacity-75 small">Open / In Progress</h6>
34                         <h2 class="display-6 fw-bold">@(stats["Open"] + stats["InProgress"])</h2>
35                     </div>
36                 </div>
37
38             <div class="col-md-3">
39                 <div class="card bg-success text-white border-0 shadow-sm">
40                     <div class="card-body">
41                         <h6 class="card-title text-uppercase opacity-75 small">Resolved</h6>
42                         <h2 class="display-6 fw-bold">@stats["Resolved"]</h2>
43                     </div>
44                 </div>
45
46             <div class="col-md-3">
47                 <div class="card bg-danger text-white border-0 shadow-sm">
48                     <div class="card-body">
49                         <h6 class="card-title text-uppercase opacity-75 small">Urgent Cases</h6>
50                         <h2 class="display-6 fw-bold">@stats["Urgent"]</h2>
51                     </div>
52                 </div>
53             </div>
54         </div>
55
56     @* --- TABLA DE GESTIÓN (MÓDULO 6) --- *@
57     <div class="card shadow-sm border-0">
58         <div class="card-header bg-white py-3">
59             <h5 class="mb-0 text-secondary">All Support Requests</h5>
60         </div>
61         <div class="table-responsive">
62             <table class="table table-hover align-middle mb-0">
63                 <thead class="table-light">
64                     <tr>
65                         <th>Title</th>
66                         <th>Author</th>
67                         <th>Category</th>
68                         <th>Status</th>
69                         <th>Technician</th>
70                         <th class="text-center">Actions</th>
71                     </tr>
72                 </thead>
73                 <tbody>
74                     @foreach (var ticket in allTickets)
75                     {
76                         <tr>
77                             <td class="fw-bold">@ticket.Title</td>
78                             <td>@ticket.Author?.Email</td>
79                             <td><span class="badge rounded-pill bg-light text-dark border">
80                                 @ticket.Category?.Name</span>
81                             </td>
82                             <td>
83                                 <span class="badge @GetStatusBadgeClass(ticket.Status)">
84                                     @ticket.Status
85                                 </span>
86                             </td>
```

```
87         <td>
88             @if (ticket.Technician != null)
89             {
90                 <span class="text-primary"><i class="bi bi-person-check"></i>
91                     @ticket.Technician.Email
92                 </span>
93             }
94             else
95             {
96                 <span class="text-muted italic">Unassigned</span>
97             }
98         </td>
99         <td class="text-center">
100             <button class="btn btn-sm btn-primary shadow-sm"
101                 @onclick="@(() => Navigation.NavigateTo($"/tickets/manage/{ticket.Id}"))">
102                 <i class="bi bi-gear-fill"></i> Manage
103             </button>
104         </td>
105     </tr>
106 }
107 </tbody>
108 </table>
109 </div>
110 </div>
111 </div>
112
113 @code {
114     private List<Ticket> allTickets = new();
115     private Dictionary<string, int> stats = new()
116     {
117         ["Total"] = 0,
118         ["Open"] = 0,
119         ["InProgress"] = 0,
120         ["Resolved"] = 0,
121         ["Urgent"] = 0
122     };
123
124     protected override async Task OnInitializedAsync()
125     {
126         // Cargar Lista
127         allTickets = await TicketService.GetAllTicketsAsync();
128
129         // Cargar Estadísticas (Asegúrate de tener este método en TicketService)
130         stats = await TicketService.GetTicketStatsAsync();
131     }
132
133     private string GetStatusBadgeClass(TicketStatus status) => status switch
134     {
135         TicketStatus.Open => "bg-info",
136         TicketStatus.InProgress => "bg-warning text-dark",
137         TicketStatus.Resolved => "bg-success",
138         _ => "bg-secondary"
139     };
140 }
141
```

Actualización

- 1. **Cards de Bootstrap 5:** Se añade una fila de tarjetas con colores semánticos (bg-primary, bg-warning, etc.) que consumen el diccionario de estadísticas.
- 2. **Iconografía:** se usa clases de Bootstrap Icons (bi-speedometer2, bi-gear-fill) para la interfaz.
- 3. **Método Helper GetStatusBadgeClass:** Se centraliza la lógica del color de los estados para que la tabla sea más fácil de leer.
- 4. **Gestión de Nulos en Tabla:** Se mejora la visualización de los técnicos no asignados con un estilo en itálica y color gris.

3. Pestaña de User Management

Esta página permite al administrador ver quién se ha registrado y asignarles el rol de *Technician* o *Employee* con un solo clic. Al igual que se hizo con el *Dashboard*, se aplica a la página de *User Management* un acabado usando **Bootstrap 5**, iconos descriptivos y una lógica de gestión de roles.

Crear el Archivo: `/Components/Pages/Admin/UserManagement.razor`

```
1  @page "/admin/users"
2  @using Microsoft.AspNetCore.Authorization
3  @using Microsoft.AspNetCore.Identity
4  @using SupportTicketsSystem2026.Data
5  @using SupportTicketsSystem2026.Models
6  @using SupportTicketsSystem2026.Services
7  @inject UserManager<ApplicationUser> UserManager
8  @inject TicketService TicketService
9  @attribute [Authorize(Roles = Roles.Admin)]
10 @rendermode InteractiveServer
11
12 <PageTitle>User Management - Ticket System</PageTitle>
13
14 <div class="container-fluid mt-4">
15     <div class="d-flex justify-content-between align-items-center mb-4 border-bottom pb-3">
16         <div>
17             <h3><i class="bi bi-people-fill text-primary"></i> User Management</h3>
18             <p class="text-muted mb-0">Manage system access and assign technical roles.</p>
19         </div>
20     </div>
21
22     @if (isLoading)
23     {
24         <div class="text-center p-5">
25             <div class="spinner-border text-primary" role="status">
26                 <span class="visually-hidden">Loading...</span>
27             </div>
28             <p class="mt-2">Cargando usuarios y roles...</p>
29         </div>
30     }
31     else
32     {
33         <div class="card shadow-sm border-0">
34             <div class="table-responsive">
35                 <table class="table table-hover align-middle mb-0">
36                     <thead class="bg-light">
37                         <tr>
38                             <th class="ps-4">User Information</th>
39                             <th>Current Role</th>
40                             <th class="text-center">Actions</th>
41                         </tr>
42                     </thead>
43                     <tbody>
44                         @foreach (var user in users)
45                         {
46                             <tr>
47                                 <td class="ps-4">
48                                     <div class="d-flex align-items-center">
49                                         <div class="avatar-circle me-3">
50                                             @user.Email?[0].ToString().ToUpper()
51                                         </div>
52                                         <div>
53                                             <div class="fw-bold text-dark">@user.Email</div>
54                                         </div>
55                                     </div>
56                                 <td>
57                                     <div>
58                                         @* Ahora leemos del diccionario, sin .Result *@
59                                         <span class="badge @GetRoleBadgeClass(userRoles[user.Id]) px-3 py-2">
60                                             @userRoles[user.Id]
61                                         </span>
62                                     </div>
63                                 <td class="text-center">
64                                     <div class="btn-group shadow-sm">
65                                         <button class="btn btn-sm btn-outline-primary"
66                                             @onclick='() => UpdateUserRole(user, Roles.Technician)'\>
67                                             <i class="bi bi-tools"></i> Make Tech
68                                         </button>
69                                         <button class="btn btn-sm btn-outline-secondary"
70                                             @onclick='() => UpdateUserRole(user, Roles.Employee)'\>
71                                             <i class="bi bi-person"></i> Make Employee
72                                         </button>
73                                     </div>
74                                 </td>
75                             </tr>
76                         }
77                     </tbody>
78                 </table>
79             </div>
80         </div>
81     }
82 }
```



```

77         </tbody>
78     </table>
79 </div>
80 </div>
81 }
82 </div>
83
84 <style>
85     .avatar-circle {
86         width: 45px;
87         height: 45px;
88         background-color: #0d6efd; /* Azul Bootstrap */
89         color: white;
90         border-radius: 50%;
91         display: flex;
92         align-items: center;
93         justify-content: center;
94         font-weight: bold;
95         font-size: 1.2rem;
96         box-shadow: 0 2px 4px rgba(0,0,0,0.1);
97         text-transform: uppercase;
98     }
99 </style>
100
101 @code {
102     private List<ApplicationUser> users = [];
103     private Dictionary<string, string> userRoles = new(); // Diccionario para evitar bloqueos
104     private bool isLoading = true;
105
106     protected override async Task OnInitializedAsync()
107     {
108         await LoadData();
109     }
110
111     private async Task LoadData()
112     {
113         isLoading = true;
114         users = await TicketService.GetAllUsersAsync();
115
116         userRoles.Clear();
117         foreach (var user in users)
118         {
119             var roles = await UserManager.GetRolesAsync(user);
120             userRoles[user.Id] = roles.FirstOrDefault() ?? "No Role";
121         }
122         isLoading = false;
123     }
124
125     private string GetRoleBadgeClass(string role) => role switch
126     {
127         Roles.Admin => "bg-dark",
128         Roles.Technician => "bg-primary",
129         Roles.Employee => "bg-secondary",
130         _ => "bg-light text-dark border"
131     };
132
133     private async Task UpdateUserRole(ApplicationUser user, string newRole)
134     {
135         // 1. Busca una instancia del usuario que el UserManager pueda trackear sin conflictos
136         var freshUser = await UserManager.FindByIdAsync(user.Id);
137
138         if (freshUser != null)
139         {
140             // 2. Obtener los roles de esa instancia fresca
141             var currentRoles = await UserManager.GetRolesAsync(freshUser);
142
143             // 3. Remover los roles antiguos
144             if (currentRoles.Any())
145             {
146                 await UserManager.RemoveFromRolesAsync(freshUser, currentRoles);
147             }
148
149             // 4. Añadir el nuevo rol
150             await UserManager.AddToRoleAsync(freshUser, newRole);
151
152             // 5. Recargar los datos para actualizar la interfaz
153             await LoadData();
154         }
155     }
156
157 }
158

```

Características:

- 1. **Avatar Dinámico:** Se crea un pequeño círculo con la inicial del email del usuario.
- 2. **Lógica de UserManager:** En el método UpdateUserRole, primero se limpian los roles anteriores (RemoveFromRolesAsync) antes de añadir el nuevo. Esto evita que un usuario termine siendo Admin y Técnico al mismo tiempo por error.
- 3. **Badges Semánticos:** Se usan los colores de las etiquetas (bg-dark para Admin, bg-primary para Tech), para coincidir con el lenguaje visual que se usa en el Dashboard.
- 4. **Botones con Iconos:** Se usan bi-tools para técnicos y bi-person para empleados que hace que la interfaz sea intuitiva.

Identity interactúa con **Blazor Server** en tiempo real. Al hacer clic en "Make Tech", la base de datos de SQL Server se actualiza instantáneamente en la tabla `AspNetUserRoles` y la UI se refresca sin recargar la página.

Último paso antes de terminar el Módulo 7

Asegurarse de que el archivo `NavMenu.razor` tenga el enlace a la página: `UserManagement`. Se actualiza el menú para que las herramientas sean accesibles solo para el perfil correcto.

Archivo: /Components/Layout/NavMenu.razor (Fragmento Admin)

```
37  @* ... el resto del código (Admin Panel, etc.) ... *@
38  @* --- SECCIÓN 3: Solo para Administradores --- *@
39  <AuthorizeView Roles="@Roles.Admin" Context="adminContext">
40      <div class="nav-item px-3">
41          <NavLink class="nav-link" href="admin/dashboard">
42              <span class="bi bi-shield-lock-fill" aria-hidden="true"></span> Dashboard
43          </NavLink>
44      </div>
45      <div class="nav-item px-3">
46          <NavLink class="nav-link" href="admin/users">
47              <span class="bi bi-people" aria-hidden="true"></span> User Management
48          </NavLink>
49      </div>
50  </AuthorizeView>
51
52  @* --- SECCIÓN 4: Gestión de Cuenta (Logout) --- *@
53  <div class="nav-item px-3">
54      <form action="Account/Logout" method="post">
55          <AntiforgeryToken />
56          <input type="hidden" name="ReturnUrl" value="" />
57          <button type="submit">
58              <span class="bi bi-arrow-bar-left-nav-menu" aria-hidden="true"></span>
59              Logout (@context.User.Identity?.Name)
60          </button>
61      </form>
62  </div>
63  </Authorized>
64
65  @* --- SECCIÓN 5: Solo para Invitados (Login/Register) --- *@
```

Resultado Final del Módulo

- 1. **Dashboard:** El Administrador ya no ve sólo una lista, ve números que indican si el departamento de soporte está al día y los trabajos pendientes. Al entrar, el administrador ve inmediatamente si hay muchos tickets "Urgentes" o "Abiertos".
- 2. **Control Total:** Ahora se puede registrar un usuario como un empleado común y, desde la aplicación, "promoverlo" a Técnico con un clic. Gestión Dinámica, ya no se necesita entrar a SQL Server para promover a un técnico. Se registra una cuenta nueva, el administrador ingresa al Panel de usuarios y le asigna el rol.
- 3. **Seguridad:** La asignación de roles usa el *UserManager* oficial de Microsoft, lo que garantiza que los cambios en la tabla `AspNetUserRoles` sean seguros y persistentes. Gracias a *UserManager*, los cambios de rol son inmediatos en la base de datos `AspNetUserRoles`.

Actualizar Repositorio (Requerido)

```
Developer PowerShell
+ Developer PowerShell v
[ ] [ ] [ ]
*****
** Visual Studio 2026 Developer PowerShell v18.4.3
** Copyright (c) 2026 Microsoft Corporation
*****
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   SupportTicketsSystem2026/Components/Layout/NavMenu.razor
        modified:   SupportTicketsSystem2026/Components/Pages/Admin/Dashboard.razor
        modified:   SupportTicketsSystem2026/Services/TicketService.cs

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        SupportTicketsSystem2026/Components/Pages/Admin/UserManagement.razor

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git add .
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   SupportTicketsSystem2026/Components/Layout/NavMenu.razor
        modified:   SupportTicketsSystem2026/Components/Pages/Admin/Dashboard.razor
        new file:   SupportTicketsSystem2026/Components/Pages/Admin/UserManagement.razor
        modified:   SupportTicketsSystem2026/Services/TicketService.cs

PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git commit -m "Mdulo 7: Implementando el Dashboard (KPIs), y la Gestión de Usuarios para administrar los permisos de los usuarios registrados"
[main cddc40f] Mdulo 7: Implementando el Dashboard (KPIs), y la Gestión de Usuarios para administrar los permisos de los usuarios registrados
4 files changed, 248 insertions(+), 13 deletions(-)
create mode 100644 SupportTicketsSystem2026/Components/Pages/Admin/UserManagement.razor
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git push origin main
Enumerating objects: 22, done.
Counting objects: 100% (22/22), done.
Delta compression using up to 24 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (12/12), 3.87 KiB | 3.87 MiB/s, done.
Total 12 (delta 8), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (8/8), completed with 8 local objects.
To https://github.com/siciuprp2020/SupportTicketsSystem2026.git
   8aca595..cddc40f  main -> main
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026>
```

Ejecutar y Probar la Aplicación

1. Asegurarse de tener registrados: el administrador, dos o más técnicos y tres o más empleados.
2. Ingresar a la aplicación como diferentes empleados y crear por lo menos 5 tickets.
3. Ingresar como **admin**:

Dashboard:

Ticket System

Home

New Ticket

My Tickets

Dashboard

User Management

Logout

←|

(admin@system.com)

About

Administrative Dashboard

Real-time Support Metrics

TOTAL TICKETS

3

OPEN / IN PROGRESS

3

RESOLVED

0

URGENT CASES

1

All Support Requests

Title	Author	Category	Status	Technician	Actions
Pantalla no se ve	empleado3@system.com	Hardware	InProgress	tecnico1@system.com	Manage
Internet Lento	empleado2@system.com	Network	InProgress	tecnico1@system.com	Manage
Programado que no funciona	empleado1@system.com	Software	InProgress	tecnico2@system.com	Manage

Dashboard, Manage:

Ticket System

Home

New Ticket

My Tickets

Dashboard

User Management

Logout

←|

(admin@system.com)

About

Manage Ticket #3

InProgress

Pantalla no se ve

Reported by: empleado3@system.com on 3/23/2026

Problemas de visualización: Pantallas negras y cuando enciende se ven colores distorsionados o falta total de señal de video.

Assign Technician

tecnico1@system.com

▼

Confirm Assignment

Update Status

Start Work

Mark Resolved

Cambiar de técnico.

Ticket System

Home

New Ticket

My Tickets

Dashboard

User Management

Logout

←|

(admin@system.com)

About

Manage Ticket #3

InProgress

Pantalla no se ve

Reported by: empleado3@system.com on 3/23/2026

Problemas de visualización: Pantallas negras y cuando enciende se ven colores distorsionados o falta total de señal de video.

Assign Technician

tecnico2@system.com

▼

Confirm Assignment

Update Status

Start Work

Mark Resolved

Ticket System

Home

New Ticket

My Tickets

Dashboard

User Management

Logout

←|

(admin@system.com)

About

Administrative Dashboard

Real-time Support Metrics

TOTAL TICKETS

3

OPEN / IN PROGRESS

3

RESOLVED

0

URGENT CASES

1

All Support Requests

Title	Author	Category	Status	Technician	Actions
Pantalla no se ve	empleado3@system.com	Hardware	InProgress	tecnico2@system.com	Manage
Internet Lento	empleado2@system.com	Network	InProgress	tecnico1@system.com	Manage
Programado que no funciona	empleado1@system.com	Software	InProgress	tecnico2@system.com	Manage

My Tickets: el administrador puede ver/administrar todos los tickets.

Ticket System

Home

New Ticket

My Tickets

Dashboard

User Management

Logout

←|

(admin@system.com)

About

My Ticket List

Title	Status	Priority	Action
Programado que no funciona	InProgress	High	Work on it
Internet Lento	InProgress	Urgent	Work on it
Pantalla no se ve	InProgress	High	Work on it

User Management: el admin, puede administrar a todos los empleados y técnicos.

Ticket System

Home

New Ticket

My Tickets

Dashboard

User Management

Logout

←|

(admin@system.com)

About

User Management

Manage system access and assign technical roles.

User Information	Current Role	Actions
A admin@system.com	Admin	Make Tech Make Employee
E empleado1@system.com	Employee	Make Tech Make Employee
E empleado2@system.com	Employee	Make Tech Make Employee
E empleado3@system.com	Employee	Make Tech Make Employee
E empleado4@system.com	No Role	Make Tech Make Employee
E empleado5@system.com	No Role	Make Tech Make Employee
T tecnico1@system.com	Technician	Make Tech Make Employee
T tecnico2@system.com	Technician	Make Tech Make Employee
T tecnico3@system.com	No Role	Make Tech Make Employee

Asignar los roles de empleado y técnico según corresponda.

Ticket System

Home

New Ticket

My Tickets

Dashboard

User Management

Logout

←|

(admin@system.com)

About

User Management

Manage system access and assign technical roles.

User Information	Current Role	Actions
A admin@system.com	Admin	Make Tech Make Employee
E empleado1@system.com	Employee	Make Tech Make Employee
E empleado2@system.com	Employee	Make Tech Make Employee
E empleado3@system.com	Employee	Make Tech Make Employee
E empleado4@system.com	Employee	Make Tech Make Employee
E empleado5@system.com	Employee	Make Tech Make Employee
T tecnico1@system.com	Technician	Make Tech Make Employee
T tecnico2@system.com	Technician	Make Tech Make Employee
T tecnico3@system.com	Technician	Make Tech Make Employee

Ingresar a la aplicación como técnico: cambiar el estatus a resuelto a dos o más tickets.

Ticket System

Home

New Ticket

My Tickets

Logout

←|

(tecnico1@system.com)

About

My Ticket List

Title	Status	Priority	Action
Internet Lento	Resolved	Urgent	Work on it

Ticket System

Home

New Ticket

My Tickets

Logout

←|

(tecnico2@system.com)

About

My Ticket List

Title	Status	Priority	Action
Programado que no funciona	InProgress	High	Work on it
Pantalla no se ve	Resolved	High	Work on it

Login como admin: My Tickets

Ticket System

Home

New Ticket

My Tickets

Dashboard

User Management

Logout

←|

(admin@system.com)

About

My Ticket List

Title	Status	Priority	Action
Programado que no funciona	InProgress	High	Work on it
Internet Lento	Resolved	Urgent	Work on it
Pantalla no se ve	Resolved	High	Work on it

Login como admin: Dashboard

Ticket System

Home

New Ticket

My Tickets

Dashboard

User Management

Logout

←|

(admin@system.com)

About

Administrative Dashboard

Real-time Support Metrics

TOTAL TICKETS3

OPEN / IN PROGRESS1

RESOLVED2

URGENT CASES1

All Support Requests

Title	Author	Category	Status	Technician	Actions
Pantalla no se ve	empleado3@system.com	Hardware	Resolved	tecnico2@system.com	Manage
Internet Lento	empleado2@system.com	Network	Resolved	tecnico1@system.com	Manage
Programado que no funciona	empleado1@system.com	Software	InProgress	tecnico2@system.com	Manage

Checkpoint

- [x] Consultas agregadas con LINQ (*Count/Sum*).
- [x] Uso de *UserManager* para manipulación dinámica de roles.
- [x] Refuerzo de *AuthorizeView* para proteger la gestión de usuarios.
- [x] UI profesional usando componentes de utilidad de Bootstrap 5.